TECNOLOGÍAS PARA LA AUTOMATIZACIÓN · 4º AÑO

2º Parcial · Robótica

Resumen teórico + ejercicios modelo resueltos paso a paso con Octave

Martes 6/10 · online | ① CI geométrica RRR | ② CI numérica DLS · cilíndrico ≤ 4 GDL | ③ Generación de trayectorias ≤ 4 GDL

Cómo usar este apunte

La **Parte A** (págs. 1–6) es la teoría justa y necesaria, en el mismo orden en que se resuelven los ejercicios. La **Parte B** son los 3 ejercicios del parcial resueltos como los tenés que hacer: primero la lógica paso a paso y después el código. Todos los números salieron de correr **las funciones del profe** (carpeta *"Archivos para el segundo parcial"*) en Octave. Al final hay **ejercicios para practicar con la respuesta**.

Antes del parcial (hacelo hoy, no el martes)

☐ Octave (o Matlab) instalado y funcionando.

☐ Una carpeta con **todas** las funciones del profe: `matrizDenavitHartenberg`, `cinematica_directa`, `cinematica_inversa`, `jacobiano_analitico`, `jacobiano_numerico`, `generacion_de_trayectorias`, `Dibujar_Temporal`, `animar_trayectoria`.

☐ Pararte en esa carpeta (o hacer `addpath('ruta/de/la/carpeta')`) y probar: `cinematica_directa([0 30 30],[0.5 0 0],[0 0.3 0.2],[90 0 0])`. Tiene que salir una matriz 4×4 con la posición del extremo en `T(1:3,4)` = [0.3598; 0; 0.8232].

☐ Tener a mano los 3 scripts modelo de este apunte (copiar, cambiar datos y correr).

Índice

Parte A · Teoría

1. El mapa: directa vs. inversa
2. Repaso express: DH y cinemática directa
3. CI geométrica del robot RRR
4. CI numérica: Jacobiano + DLS
5. El robot cilíndrico
6. Generación de trayectorias
7. Las funciones del profe (chuleta)

Parte B · Ejercicios modelo

Ejercicio 1 · RRR por geometría

Ejercicio 2 · Cilíndrico por DLS (a completar)

Ejercicio 3 · Trayectoria (con código)

Para practicar (con respuestas)

Errores típicos que restan puntos

1 El mapa: directa vs. inversa

Un robot tiene dos "idiomas": el **espacio articular** (los valores de cada articulación: ángulos `q` en las rotativas y desplazamientos `d` en las prismáticas) y el **espacio cartesiano** (dónde está la punta o TCP: `x, y, z`).

Cinemática directa (CD)

q1, q2, q3... → x, y, z

Siempre tiene **una sola** respuesta. Se resuelve multiplicando matrices DH (1º parcial).

Cinemática inversa (CI)

x, y, z → q1, q2, q3...

Puede tener **varias** soluciones (codo arriba/abajo), una sola o **ninguna** (punto fuera de alcance).

Para la CI hay dos caminos, y en el parcial se toma uno en cada ejercicio:

	Geométrica (Ej. 1)	Numérica DLS (Ej. 2)
Cómo	Trigonometría en el dibujo: `atan2` + ley del coseno.	Iterando: arranca en una semilla `q0` y corrige con el Jacobiano hasta llegar.
Resultado	Fórmula cerrada, exacta, te da **todas** las soluciones.	Aproximado (error < tolerancia), te da **la solución más cercana a la semilla**.
Sirve para	Robots simples (2R, 3R).	Cualquier robot, con articulaciones R y P.

Y la **generación de trayectorias** (Ej. 3) usa todo lo anterior: pasar de un punto A a un punto B generando muchas posturas intermedias en el tiempo.

2 Repaso express: DH y cinemática directa

Cada fila de la tabla de Denavit–Hartenberg describe cómo pasar del sistema *i–1* al sistema *i* con 4 movimientos, **siempre en este orden**:

$$A_i = Rot_z(\theta_i) \cdot Tras_z(d_i) \cdot Tras_x(a_i) \cdot Rot_x(\alpha_i)$$

$$A_i = \begin{vmatrix} \cos\theta & -\text{sen}\theta\cdot\cos\alpha & \text{sen}\theta\cdot\text{sen}\alpha & a\cdot\cos\theta \\ \text{sen}\theta & \cos\theta\cdot\cos\alpha & -\cos\theta\cdot\text{sen}\alpha & a\cdot\text{sen}\theta \\ 0 & \text{sen}\alpha & \cos\alpha & d \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

θ: giro alrededor de z_{i-1} · **d**: avance sobre z_{i-1}
a: avance sobre x_i · **α**: giro alrededor de x_i
Articulación **R** → la variable es **θ**.
Articulación **P** → la variable es **d**.

Cinemática directa

$${}^{0}T_n = A_1 \cdot A_2 \cdot \ldots \cdot A_n \rightarrow \text{posición del TCP} = \texttt{T(1:3,4)} \cdot \text{orientación} = \texttt{T(1:3,1:3)}$$

En Octave: `T = cinematica_directa(theta, d, a, alpha, 'deg')` (los 4 vectores son las 4 columnas de la tabla DH ya con los valores numéricos).

El robot RRR (antropomorfo) que usa la cátedra

i	θ	d	a	α
1	q1	l1	0	90°
2	q2	0	l2	0
3	q3	0	l3	0

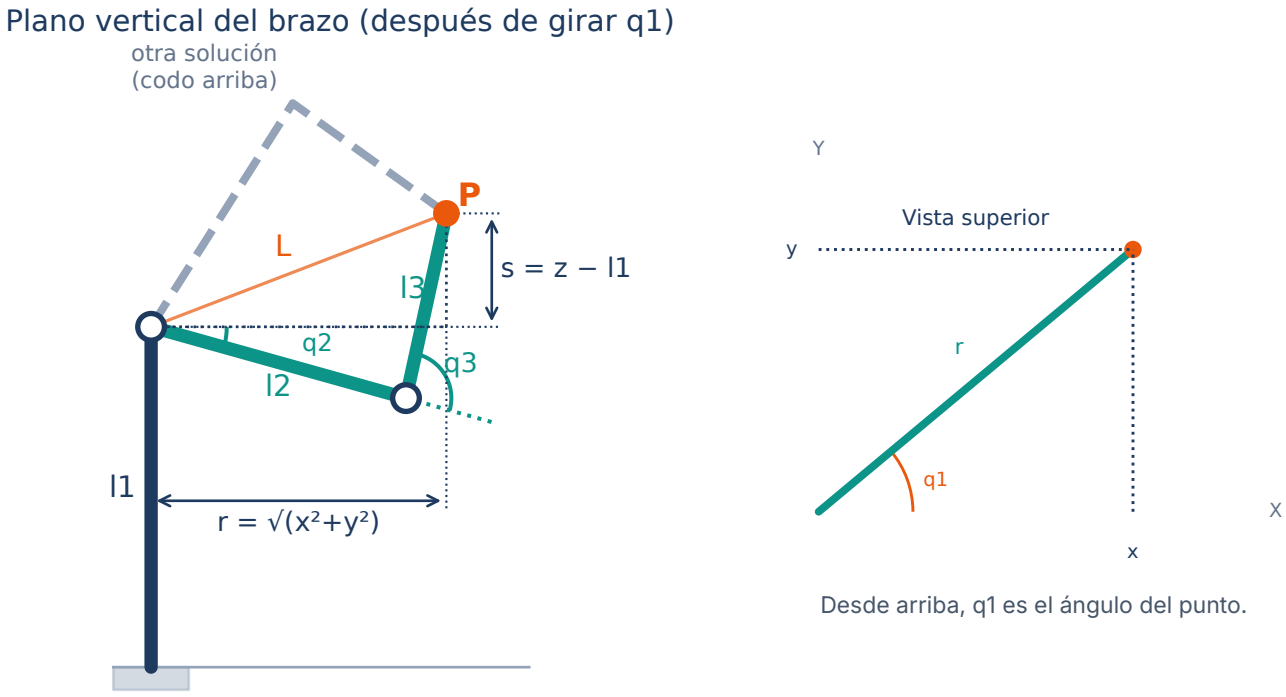
q1 gira la base alrededor de Z.
q2 se mide desde la **horizontal** hasta l2.
q3 = 0 cuando l3 es la **prolongación** de l2 (es un ángulo relativo).

Multiplicando las tres matrices queda la CD en forma cerrada (la vas a usar para verificar):

$$x = \cos q1 \cdot [\, l2 \cdot \cos q2 + l3 \cdot \cos(q2+q3) \,] \qquad y = \operatorname{sen} q1 \cdot [\, l2 \cdot \cos q2 + l3 \cdot \cos(q2+q3) \,]$$
$$z = l1 + l2 \cdot \operatorname{sen} q2 + l3 \cdot \operatorname{sen}(q2+q3)$$

3 CI geométrica del robot RRR

La idea: el problema 3D se parte en **dos problemas 2D**. Mirando desde arriba sale q1; después "parás" al robot en su plano vertical y queda un brazo 2R plano, que se resuelve con la ley del coseno.



Desde arriba, q1 es el ángulo del punto.

Brazo 2R en el plano vertical (datos del Ejercicio 1).

1. **Giro de la base.** Desde arriba el brazo apunta al punto: $q1 = \operatorname{atan2}(y, x)$. Usá siempre `atan2d(y,x)` (no `atand(y/x)`): `atan2` reconoce el cuadrante.
2. **Bajo al plano del brazo.** Distancia horizontal y altura medidas desde el hombro:
$$r = \sqrt{x^2 + y^2} \qquad s = z - l1$$
3. **¿Llega?** La distancia hombro–punto es $L = \sqrt{r^2 + s^2}$. Tiene que cumplirse $|l2 - l3| \le L \le l2 + l3$. Si no, el punto está fuera de alcance (la función del profe tira error).

4. **Ley del coseno → q3.** En el triángulo hombro–codo–punto:

$$\cos q3 = D = (r^2 + s^2 - l2^2 - l3^2) / (2 \cdot l2 \cdot l3)$$

$$q3 = \operatorname{atan2}(\pm\sqrt{(1 - D^2)}, D)$$

El ± da las **dos soluciones**. Si $|D| > 1$ → no hay solución (fuera de alcance).

5. **q2** = (ángulo hacia el punto) − (ángulo que "come" el codo):

$$q2 = \operatorname{atan2}(s, r) - \operatorname{atan2}(l3 \cdot \operatorname{sen} q3 , l2 + l3 \cdot \cos q3)$$

Se calcula **una vez por cada q3** (cada q3 tiene su q2).

6. **Verificar** metiendo (q1, q2, q3) en la cinemática directa: tiene que volver a dar (x, y, z). Es la forma de asegurarte en el parcial de que no te equivocaste de signo.

Las dos soluciones y cómo las nombra el profe

Con q3 > 0 el eslabón l3 "sube" respecto de la prolongación de l2: el codo queda **abajo**. Con q3 < 0 el codo queda **arriba**. En la rutina del profe (`cinematica_inversa_3R.m`) la variable `use_elbow = "up"` elige **q3 positivo**. Si el enunciado pide "codo arriba/abajo", fijate qué convención usa; si no aclara, informá las dos.

Dato extra: existen otras dos soluciones "mirando para atrás" (q1+180°, 180°−q2, −q3), pero la cátedra trabaja con las dos de frente.

¿De dónde sale la fórmula de q2? (por si te la piden explicar)

β = atan2(s, r) es el ángulo de la recta hombro→punto respecto de la horizontal. El codo hace que l2 no apunte directo al punto: se desvía un ángulo γ = atan2(l3·sen q3, l2 + l3·cos q3), que es el ángulo del vector "l2 + l3 girado q3". Entonces q2 = β − γ.

4 CI numérica: Jacobiano + DLS

Cuando el robot es más complicado (o mezcla R y P) no se busca una fórmula: se **prueba, se mide el error y se corrige**, muchas veces, hasta llegar. La herramienta que dice "cuánto corregir" es el Jacobiano.

El Jacobiano J

Es la matriz de derivadas de la posición respecto de cada articulación. Dice cuánto se mueve la punta si muevo un poquito cada articulación:

$$\Delta P \approx J \cdot \Delta q \qquad J = [\ \partial x/\partial q1 \ \ \partial x/\partial q2 \ \ldots\ ;\ \partial y/\partial q1 \ \ldots\ ;\ \partial z/\partial q1 \ \ldots\ ]\ \ (3 \times n)$$

Cada **columna** es el movimiento de la punta cuando se mueve **una sola** articulación. Así la arma `jacobiano_analitico`:

Articulación i	Columna de posición (J_v)	Interpretación
Rotativa (R)	$z_{i-1} \times (p_n - p_{i-1})$	Al girar, la punta se mueve perpendicular al eje y al "brazo de palanca".
Prismática (P)	z_{i-1}	Al deslizar, la punta se mueve en la dirección del eje.

Grados: el factor π/180

Las derivadas son "por radián". Como las funciones trabajan en grados, las columnas de las **rotativas** se multiplican por **π/180 ≈ 0,017453**. Ejemplo del Ejercicio 2: con brazo radial de 200 mm, la columna de q1 vale 200·π/180 = **3,4907 mm por grado**. Las columnas de las prismáticas no se tocan (mm por mm).

La corrección DLS (Damped Least Squares, mínimos cuadrados amortiguados)

Lo ideal sería $\Delta q = J^{-1} \cdot e$, pero J puede no ser cuadrada (robot de 4 GDL → J es 3×4) o estar cerca de una **singularidad** (brazo estirado, punto sobre el eje Z) y ahí la inversa "explota". DLS lo resuelve:

$$\Delta q = J^T \cdot (J \cdot J^T + \lambda^2 \cdot I)^{-1} \cdot e$$

en Octave: `dq = Jp' * ((Jp*Jp' + lambda^2*eye(3)) \ e);`

Minimiza $\|J \cdot \Delta q - e\|^2 + \lambda^2 \cdot \|\Delta q\|^2$: reducir el error **sin** hacer correcciones exageradas. λ = 0 sería la pseudoinversa; λ más grande = más estable pero más lento.

El ciclo completo (lo que hace `cinematica_inversa` por dentro)

q (empieza en q0) → CD → p → e = pd − p → ¿‖e‖ < tol? → FIN → J(q) → Δq por DLS → q = q + α·Δq ↺

Parámetro	Valor por defecto (función del 2º parcial)	Qué hace
`lambda`	0.05	Amortiguamiento. Estabiliza cerca de singularidades.
`tol_pos`	1e-3 (en las unidades de pd → **mm**)	Cuándo se considera que llegó.
`max_iter`	200	Si no llega en 200 vueltas, devuelve `success = false`.
`alpha_step`	1.0	Fracción de Δq que se aplica en cada vuelta.
`q_min`, `q_max`	sin límites	Límites articulares (muy útiles para las prismáticas, ver ej. 2).

Tres cosas que tenés que saber de la CI numérica

1. **La semilla q0 manda:** la función converge a la solución "más cercana" a q0. Distinta semilla → puede dar otra solución (ej. codo arriba en vez de abajo).
2. **Unidades:** trabajá en **mm** (la tolerancia 1e-3 está pensada en mm). Si trabajás en metros, 1e-3 m = 1 mm es muy grosero → pasá `opts.tol_pos = 1e-6`.
3. **Siempre verificá** con la CD que el punto obtenido es el pedido y que los valores tienen sentido físico (una prismática no puede ser negativa).

5 El robot cilíndrico

Una rotación en la base y dos prismáticas: una **vertical** (sube/baja) y una **radial** (estira/encoge el brazo). Su espacio de trabajo es un cilindro hueco. La versión de 4 GDL (la del apunte de Denavit–Hartenberg de la cátedra) le agrega un giro de muñeca al final.

i	tipo	θ	d	a	α
1	R	**q1**	l1	0	0
2	P	90°	**d2**	0	90°
3	P	0	**d3**	0	0
4	R	**q4**	l4	0	0

Variables en negrita. Para la versión de **3 GDL (RPP)** se borra la fila 4.

CD en forma cerrada

$$x = (d3 + l4)\cdot\cos q1$$
$$y = (d3 + l4)\cdot\text{sen } q1$$
$$z = l1 + d2$$

q4 **no** cambia la posición (gira la muñeca sobre su eje): solo cambia la orientación.

Cómo se carga en `cinematica_inversa` (la parte que más se confunde)

La función arma θ y d así: **R** → `θ = q_offset + q` (y d sale de `d_offset`); **P** → `d = d_offset + q` (y θ sale de `q_offset`). Entonces la regla es:

Regla práctica

Copiá las columnas θ y d de la tabla DH. **Donde hay una variable poné 0** (o el offset si la variable aparece como "q + algo"). Lo demás queda con su valor fijo.

```
tipo     = 'RPPR';
q_offset = [0;  90; 0; 0];     % columna θ: q1→0, 90 fijo, 0 fijo, q4→0   [deg]
d_offset = [l1;  0; 0; l4];    % columna d: l1 fijo, d2→0, d3→0, l4 fijo  [mm]
a        = [0; 0; 0; 0];
alpha    = [0; 90; 0; 0];      % [deg]
q0       = [q1; d2; d3; q4];   % semilla: grados para las R, mm para las P
```

⚠ La versión del 2º parcial devuelve 5 salidas

La `cinematica_inversa.m` de la carpeta del 2º parcial es `[q_sol, theta, d, success, info]` (la de la carpeta "Funciones" devuelve solo 3). Si escribís `[q, ok] = cinematica_inversa(...)`, en `ok` te queda **theta**, no el éxito. Usá siempre las 5:

```
[q_sol, theta, d, success, info] = cinematica_inversa(pd, q0, tipo, q_offset, d_offset, a,
alpha, 'deg');
```

⚠ La trampa del cilíndrico: prismáticas negativas

Si la semilla tiene q1 lejos del ángulo del punto, DLS puede converger a una solución **matemáticamente correcta pero físicamente imposible**: girar 180° de más y estirar el brazo con **d3 negativo** (hacia atrás). Comprobado con las funciones del profe: para pd = (250, −250, 500) con q0 = [0; 100; 100] da q1 = 135°, d3 = −353,55 ✗. Dos soluciones (las dos verificadas):

- **Buena semilla:** poné q1 de la semilla ≈ `atan2d(y,x)` → q1 = −45°, d3 = 353,55 en 4 iteraciones ✓.
- **Límites:** `opts.q_min = [-180; 0; 0]; opts.q_max = [180; 600; 600];` y pasá `opts` como último argumento → q1 = −45°, d3 = 353,55 ✓.

6 Generación de trayectorias

Una trayectoria es una **sucesión de posturas en el tiempo**: una matriz donde cada **fila** es la postura completa del robot en un instante (`q(k,:)` en el tiempo `t(k)`). Hay dos formas de generarla:

A) En el espacio articular: `generacion_de_trayectorias(qi, qf, wmax, h)`

Movimiento punto a punto **sincronizado**: todas las articulaciones arrancan y llegan juntas, cada una a velocidad constante.

1. Recorrido de cada eje: $\Delta q = qf - qi$
2. Tiempo total = el de la articulación **más lenta** (la que más tarda yendo a su velocidad máxima): $T = \max(\ |\Delta q_k| \ / \ wmax_k\ )$
3. Velocidad de cada eje para terminar justo en T: $w_k = \Delta q_k \ / \ T$ (la más lenta va a su wmax, el resto más despacio).
4. Cantidad de puntos y tiempos: $n = \max(2,\ \lceil T/h \rceil + 1)$, `t = linspace(0, T, n)` (paso real ≤ h).
5. Posición lineal en el tiempo: $q(t) = qi + w \cdot t$ y velocidad constante `Dq = w`.

Salidas: `q` (n × GDL), `Dq` (n × GDL, todas las filas iguales), `t` (n × 1). Las unidades son las que le des: grados y grados/s para las R, mm y mm/s para las P (se pueden mezclar en el mismo vector).

Ojo con lo que NO hace

Que las articulaciones se muevan en línea recta **no** significa que la punta se mueva en línea recta: el TCP describe una curva (ver la figura del Ejercicio 3). Además el perfil de velocidad es rectangular (arranca y frena de golpe).

B) En el espacio cartesiano: recta o círculo + CI punto por punto

Si se pide que la **punta** haga una recta o un círculo, primero se generan los puntos cartesianos y después se hace la CI de **cada uno**:

Recta de P1 a P2

$$P(s) = P1 + s \cdot (P2 - P1), \quad s \in [0,\ 1]$$

s = 0 da P1, s = 1 da P2. Con `s = 0:0.01:1` salen 101 puntos.

Círculo vertical (plano XZ)

$$P(w) = (\ xc + R \cdot \cos w,\ \ yc,\ \ zc + R \cdot \text{sen}\ w\ )$$

w de 0° a 360°. Con paso 5° salen 73 puntos (el primero = el último).

Truco clave: encadenar semillas

En el `for`, la semilla de cada punto es **la solución del punto anterior**. Como los puntos están cerca, DLS converge en 2–3 iteraciones y el robot no "salta" de una solución a otra (continuidad).

Para ver los resultados

`Dibujar_Temporal(t, q, Dq)` grafica posiciones y velocidades en el tiempo. `animar_trayectoria(t, q, dh_params, tipo)` anima el robot en 3D (ver detalles en la chuleta).

7 Las funciones del profe (chuleta)

Llamada	Entradas	Salidas / para qué
`A = matrizDenavitHartenberg(q, d, a, alfa, 'deg')`	Una fila de la tabla DH	Matriz 4×4 A_i
`[T, origins, Tsist] = cinematica_directa(theta, d, a, alpha, 'deg')`	Las 4 columnas DH con valores numéricos	`T`: 4×4 del TCP. `origins`: 3×(n+1) con cada origen (base…TCP). `Tsist`: las A_i.
`[q_sol, theta, d, success, info] = cinematica_inversa(pd, q0, tipo, q_offset, d_offset, a, alpha, 'deg', opts)`	`pd` [mm], semilla `q0`, `tipo` ('RPPR'…), tabla DH en forma de offsets. `opts` opcional.	`q_sol`: solución. `success`: 1 si llegó. `info.iters`: iteraciones. `info.err_pos`: error final.
`J = jacobiano_analitico(q, tipo, q_offset, d_offset, a, alpha, 'deg')`	Igual que la CI	J de 6×n. Filas 1–3: posición (las que usa DLS). Filas 4–6: orientación.
`J = jacobiano_numerico(...)`	Igual	Lo mismo por diferencias finitas (sirve para comparar).
`[q, Dq, t] = generacion_de_trayectorias(qi, qf, wmax, h)`	Vectores fila; `wmax` por eje; `h` en s	Posiciones, velocidades y tiempos.
`Dibujar_Temporal(t, q, Dq)`	Salidas de la anterior	Gráficos de posición y velocidad.
`animar_trayectoria(t, q, dh_params, tipo)`	`dh_params` = n×3: `[a, alpha, d_o_θ_fijo]`	Animación 3D del robot.

Detalles de `animar_trayectoria` (la del 2º parcial)

- La 3ª columna de `dh_params` es el valor **fijo** de cada fila: **d** si la articulación es R, **θ** si es P.
- `alpha` y los θ fijos van en **radianes** (usa `cos`, no `cosd`).
- Pasale las columnas R de `q` en radianes (`deg2rad`): la función solo convierte sola si algún valor supera 2π, y con movimientos chicos no lo hace.
- No admite offsets en las R (toma θ = q) ni en las P (toma d = q).

B Ejercicio 1 · CI geométrica de un robot RRR

Enunciado modelo

Un robot antropomorfo RRR tiene la tabla DH de la sección 2, con l1 = 0,45 m, l2 = 0,35 m y l3 = 0,25 m. Calcular por el **método geométrico** los valores articulares (q1, q2, q3) que llevan el extremo al punto **P = (0,30; 0,25; 0,60) m**. Informar ambas soluciones y verificar.

1. **q1** = atan2(0,25; 0,30) = **39,806°**
2. r = √(0,30² + 0,25²) = **0,39051 m** · s = 0,60 − 0,45 = **0,15 m**
3. Alcance: L = √(0,39051² + 0,15²) = 0,41833 m. Como 0,10 ≤ 0,41833 ≤ 0,60 → **llega** ✓
4. D = (0,39051² + 0,15² − 0,35² − 0,25²) / (2·0,35·0,25) = (0,1750 − 0,1850) / 0,175 = **−0,057143**
   q3 = atan2(±√(1 − 0,057143²), −0,057143) = **±93,276°**
5. β = atan2(0,15; 0,39051) = 21,0167°
   Con q3 = +93,276°: γ = atan2(0,25·sen 93,276°; 0,35 + 0,25·cos 93,276°) = 36,634° → q2 = 21,017 − 36,634 = **−15,617°**
   Con q3 = −93,276°: γ = −36,634° → q2 = 21,017 + 36,634 = **57,642°**

Solución	q1	q2	q3	Codo (origen 2) en z
A · q3 > 0 (“up” en la rutina del profe)	39,806°	−15,617°	93,276°	0,356 m → codo **abajo**
B · q3 < 0	39,806°	57,642°	−93,276°	0,746 m → codo **arriba**

6. **Verificación** con la CD del profe: las dos soluciones dan `T(1:3,4) = [0.3; 0.25; 0.6]` ✓

Código (copiar, cambiar datos y correr)

```
l1 = 0.45; l2 = 0.35; l3 = 0.25;            % [m]
P  = [0.30 0.25 0.60];  x = P(1); y = P(2); z = P(3);

q1 = atan2d(y, x)                            % 1) giro de la base
r  = hypot(x, y);   s = z - l1;              % 2) plano del brazo
L  = hypot(r, s);
if L > l2 + l3 || L < abs(l2 - l3), error('Fuera de alcance'); end
D  = (r^2 + s^2 - l2^2 - l3^2) / (2*l2*l3)   % 3) ley del coseno
q3A =  atan2d(sqrt(1-D^2), D);               % solucion A (q3 > 0)
q3B = -atan2d(sqrt(1-D^2), D);               % solucion B (q3 < 0)
q2A = atan2d(s, r) - atan2d(l3*sind(q3A), l2 + l3*cosd(q3A));
q2B = atan2d(s, r) - atan2d(l3*sind(q3B), l2 + l3*cosd(q3B));
solA = [q1 q2A q3A], solB = [q1 q2B q3B]

% 4) verificacion con la CD (tabla DH del RRR)
d = [l1 0 0];  a = [0 l2 l3];  alpha = [90 0 0];
TA = cinematica_directa(solA, d, a, alpha, 'deg');  PA = TA(1:3,4)'
TB = cinematica_directa(solB, d, a, alpha, 'deg');  PB = TB(1:3,4)'
```

```
q1 = 39.806     D = -0.057143
solA =   39.806  -15.617   93.276
solB =   39.806   57.642  -93.276
PA =   0.3   0.25   0.6
PB =   0.3   0.25   0.6
```

B Ejercicio 2 · CI numérica (DLS) de un robot cilíndrico

Enunciado modelo (formato "resultados a completar")

El robot cilíndrico de 4 GDL de la sección 5 tiene l1 = 400 mm y l4 = 100 mm. Usando `cinematica_inversa` (DLS, parámetros por defecto) con la semilla **q0 = [0°; 100 mm; 100 mm; 0°]**, hallar la postura que lleva el TCP a **pd = (300; 250; 650) mm**. Completar: posición inicial, error inicial, Jacobiano en q0, Δq y postura después de la 1ª iteración, solución final, cantidad de iteraciones y error final.

Paso 1 · Cargar el robot (tabla DH → vectores)

```
l1 = 400;  l4 = 100;                    % [mm]
tipo     = 'RPPR';
q_offset = [0; 90; 0; 0];               % [deg]
d_offset = [l1; 0; 0; l4];              % [mm]
a        = [0; 0; 0; 0];
alpha    = [0; 90; 0; 0];               % [deg]
pd = [300; 250; 650];                   % [mm]
q0 = [0; 100; 100; 0];                  % [deg; mm; mm; deg]
```

Paso 2 · Iteración 1 "a mano" (lo que hace la función por dentro)

a **CD en q0:** θ = [0; 90; 0; 0], d = [400; 100; 100; 100] → p0 = ((100+100)·cos 0; (100+100)·sen 0; 400+100) = **(200; 0; 500) mm**

b **Error:** e0 = pd − p0 = **(100; 250; 150) mm** → ‖e0‖ = **308,22 mm**

c **Jacobiano de posición en q0** (filas 1–3 de `jacobiano_analitico`):

	q1	d2	d3	q4
x	0	0	1	0
y	3,4907	0	0	0
z	0	1	0	0

- Columna q1: girar 1° mueve la punta 200·π/180 = 3,4907 mm en **y** (tangente al giro).
- d2 mueve en **z**, d3 mueve en **x** (radial con q1 = 0).
- q4 = columna de ceros: la muñeca no mueve la posición.

d **DLS:** $\Delta q = J^T(JJ^T + 0{,}05^2 \cdot I)^{-1} \cdot e0$ = **(71,605°; 149,63 mm; 99,751 mm; 0°)**

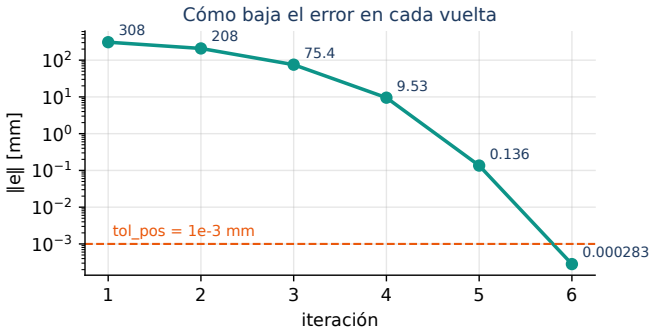
Fijate que q1 se pasa de largo (71,6° cuando la respuesta es 39,8°): el Jacobiano es lineal y el giro no. Por eso hacen falta varias vueltas.

e **Postura tras la iteración 1:** q1 = q0 + Δq = **(71,605°; 249,63 mm; 199,75 mm; 0°)**

Paso 3 · La función completa

```
[q_sol, theta, d, success, info] = cinematica_inversa(pd, q0, tipo, q_offset, d_offset, a, alpha,
'deg')

% verificacion: CD con la solucion
th = q_offset + [q_sol(1); 0; 0; q_sol(4)];
dd = d_offset + [0; q_sol(2); q_sol(3); 0];
T  = cinematica_directa(th, dd, a, alpha, 'deg');
p_final = T(1:3,4),  error_final = norm(pd - p_final)
```



iter	‖e‖ [mm]	q = [q1 d2 d3 q4]
1	308,22	0 · 100 · 100 · 0
2	208,28	71,61 · 249,63 · 199,75 · 0
3	75,370	32,27 · 250 · 231,82 · 0
4	9,5295	41,11 · 250 · 287,01 · 0
5	0,13559	39,79 · 250 · 290,40 · 0
6	0,00028	39,81 · 250 · 290,51 · 0 ✓

(Para ver esta tabla en Octave: `opts.verbose = true;` y pasar `opts` como último argumento.)

Resultados a completar

Pregunta · Completar con los valores obtenidos (2 decimales)

Posición inicial p0 = (200 ; 0 ; 500) mm · ‖e0‖ = 308.22 mm
Tras la 1ª iteración: q1 = 71.61 ° d2 = 249.63 mm d3 = 199.75 mm
Solución: q1 = 39.81 ° d2 = 250.00 mm d3 = 290.51 mm q4 = 0 °
Iteraciones (`info.iters`) = 6 · success = 1 · error final = 2.83e-4 mm

Control rápido sin Octave (geometría)

q1 = atan2(250, 300) = 39,806° ✓ · d2 = 650 − 400 = 250 ✓ · d3 = √(300² + 250²) − 100 = 390,51 − 100 = 290,51 ✓. Si DLS te da algo distinto, revisá la carga de offsets.

B Ejercicio 3 · Generación de trayectoria (se responde con código)

Enunciado modelo

Con el robot cilíndrico del Ejercicio 2, generar una trayectoria articular sincronizada que lleve el TCP de **A = (300; 250; 650) mm** a **B = (−200; 350; 500) mm**. Velocidades máximas: 30 °/s (q1), 50 mm/s (d2), 50 mm/s (d3), 30 °/s (q4). Paso de muestreo h = 0,1 s. Informar el tiempo total, la cantidad de puntos, las velocidades de cada eje, graficar y verificar que empieza en A y termina en B.

La lógica (3 pasos)

A y B (cartesiano) → CI de cada uno → qA, qB → generacion_de_trayectorias → CD de cada fila: verificar

```
%% EJERCICIO 3 - Trayectoria articular sincronizada (cilindrico 4 GdL)
clc; clear;
l1 = 400; l4 = 100;
tipo = 'RPPR';  q_offset = [0;90;0;0];  d_offset = [l1;0;0;l4];
a = [0;0;0;0];  alpha = [0;90;0;0];
PA = [300; 250; 650];   PB = [-200; 350; 500];          % [mm]

% 1) CI de los dos puntos (la semilla de B es la solucion de A)
[qA,~,~,okA] = cinematica_inversa(PA, [0;100;100;0], tipo, q_offset, d_offset, a, alpha, 'deg');
[qB,~,~,okB] = cinematica_inversa(PB, qA,            tipo, q_offset, d_offset, a, alpha, 'deg');
qA = qA',  qB = qB'                                      % como fila

% 2) trayectoria articular sincronizada
wmax = [30 50 50 30];               % [deg/s  mm/s  mm/s  deg/s]
h    = 0.1;                         % [s]
[q, Dq, t] = generacion_de_trayectorias(qA, qB, wmax, h);
T_total = t(end),  N_puntos = size(q,1),  velocidades = Dq(1,:)

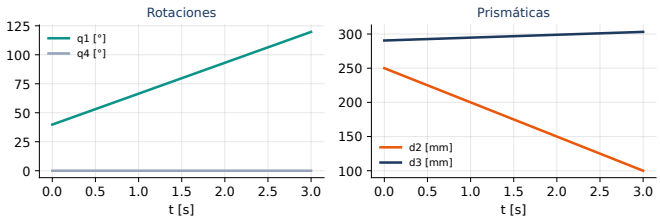
% 3) verificar: CD de cada fila -> camino del TCP
P = zeros(3, N_puntos);
for k = 1:N_puntos
  th = q_offset + [q(k,1); 0; 0; q(k,4)];
  dd = d_offset + [0; q(k,2); q(k,3); 0];
  Tk = cinematica_directa(th, dd, a, alpha, 'deg');
  P(:,k) = Tk(1:3,4);
end
P_inicio = P(:,1)',  P_fin = P(:,end)'

% 4) graficos y animacion
Dibujar_Temporal(t, q, Dq);
qr = q;  qr(:,[1 4]) = deg2rad(q(:,[1 4]));            % las R en radianes
dh = [0 0 l1; 0 pi/2 pi/2; 0 0 0; 0 0 l4];              % [a  alpha  fijo]
animar_trayectoria(t, qr, dh, 'RPPR');
```

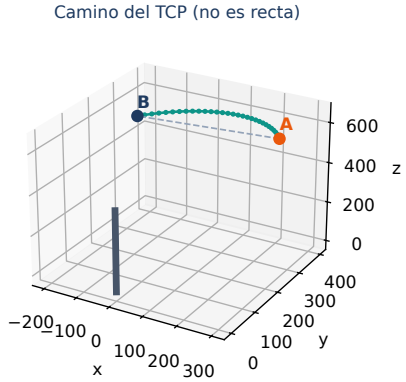
```
qA =  39.806   250   290.51   0
qB = 119.74    100   303.11   0
T_total = 3      N_puntos = 31
velocidades =  26.646  -50   4.2002   0
P_inicio =   300   250   650
P_fin    =  -200   350   500
```

Cómo se explica el resultado

1. Δq = qB − qA = (79,93°; −150 mm; 12,60 mm; 0°)
2. Tiempo de cada eje a su velocidad máxima: 79,93/30 = 2,66 s · 150/50 = **3 s** · 12,6/50 = 0,25 s · 0 → manda **d2**: **T = 3 s**.
3. Velocidades ecualizadas w = Δq/T = (26,646 °/s; −50 mm/s; 4,2 mm/s; 0). Solo d2 va a su máximo.
4. n = [3/0,1] + 1 = **31 puntos** (t = 0; 0,1; …; 3 s). En el medio (k = 16, t = 1,5 s): q = (79,775°; 175; 296,81; 0).



Cada articulación cambia linealmente con el tiempo.



El TCP va de A a B por una curva (gris = la recta).

Variante 1 · Te dan qi y qf directamente (RRR)

```
qi = [0 30 30];  qf = [90 60 -30];          % [deg]
wmax = [30 20 40];  h = 0.1;
[q, Dq, t] = generacion_de_trayectorias(qi, qf, wmax, h);
t(end), size(q,1), Dq(1,:)                   % -> 3 s, 31 puntos, [30 10 -20] deg/s
Dibujar_Temporal(t, q, Dq);
```

Variante 2 · Que la punta vaya en línea recta (CI punto a punto)

```
s = linspace(0, 1, 51);                      % 51 puntos de A a B
Q = zeros(numel(s), 4);  qk = qA';           % semilla = solucion de A
for k = 1:numel(s)
  Pk = PA + s(k)*(PB - PA);                  % punto de la recta
  [qk,~,~,ok] = cinematica_inversa(Pk, qk, tipo, q_offset, d_offset, a, alpha, 'deg');
  Q(k,:) = qk';                              % la semilla del siguiente es esta
end
```

✎ Para practicar (con respuestas)

Todas verificadas con las funciones del profe. Intentá resolverlas antes de mirar la respuesta.

Práctica 1 · CI geométrica (datos de la clase)

RRR con l1 = 0,5 m, l2 = 0,3 m, l3 = 0,2 m. Punto P = (0,3598; 0; 0,8232) m.

Respuesta: q1 = `0` ° · Solución A: q2 = `30.00` °, q3 = `30.01` ° · Solución B: q2 = `53.87` °, q3 = `-30.01` °

(Es el punto que sale de la CD con q = (0, 30, 30); la diferencia de centésimas es por el redondeo de P.)

Práctica 2 · CI geométrica con el punto en el 2º cuadrante

RRR con l1 = 0,4 m, l2 = 0,3 m, l3 = 0,3 m. Punto P = (−0,2; 0,35; 0,5) m.

Respuesta: r = 0,40311 · s = 0,1 · D = −0,041667 · q1 = `119.74` °

Solución A: q2 = `-32.26` °, q3 = `92.39` ° · Solución B: q2 = `60.13` °, q3 = `-92.39` °

(Con atand(y/x) te daría −60,26°: por eso se usa atan2.)

Práctica 3 · DLS cilíndrico 4 GDL, buena semilla

Mismo robot del Ej. 2. pd = (−150; 200; 550) mm, q0 = [90; 50; 50; 0].
Respuesta: q1 = `126.87` ° · d2 = `150` mm · d3 = `150` mm · q4 = `0` · iteraciones = `6` · success = 1

Práctica 4 · La misma, con semilla mala

pd = (−150; 200; 550) mm pero con q0 = [0; 100; 100; 0].
Respuesta: converge (success = 1, 11 iteraciones) a q1 = −53,13°, d3 = **−350 mm ✗** (imposible). Con `opts.q_min = [-180;0;0;-180]`, `opts.q_max = [180;600;600;180]` da la correcta: `126.87` °, 150, 150, 0 en 8 iteraciones.

Práctica 5 · Cilíndrico de 3 GDL (RPP)

l1 = 300 mm, sin muñeca (`tipo = 'RPP'`, `q_offset = [0;90;0]`, `d_offset = [300;0;0]`, `a = [0;0;0]`, `alpha = [0;90;0]`). pd = (250; −250; 500) mm, q0 = [−45; 100; 100].
Respuesta: q1 = `-45` ° · d2 = `200` mm · d3 = `353.55` mm · iteraciones = `4`

Práctica 6 · Trayectoria cuando h no divide justo a T

qi = [10 0 50], qf = [−50 40 20] (grados), wmax = [25 25 25] °/s, h = 0,15 s.
Respuesta: T = `2.4` s (manda q1: 60/25) · puntos = `17` · paso real = `0.15` s · velocidades = `-25`, `16.67`, `-12.5` °/s

! Errores típicos que restan puntos

Error	Cómo evitarlo
Usar `atand(y/x)` para q1	Siempre `atan2d(y, x)`.
Mezclar grados y radianes (`sin` con grados)	Con grados: `sind`, `cosd`, `atan2d`.
Calcular un solo q2 para las dos soluciones	Cada q3 tiene su propio q2.
Olvidarse de restar l1 (s = z, en vez de z − l1)	La ley del coseno se plantea desde el **hombro**.
`[q, ok] = cinematica_inversa(...)`	La del 2º parcial tiene 5 salidas: `[q_sol, theta, d, success, info]`.
Poner la variable en `q_offset` / `d_offset`	Donde hay variable va 0 (o su offset).
Trabajar en metros con tol = 1e-3	Usá mm, o `opts.tol_pos = 1e-6`.
Aceptar una prismática negativa	Mejor semilla (q1 ≈ atan2d(y,x)) o límites q_min/q_max.
Pasar qi/qf como columna a la trayectoria	La función los fuerza a fila, pero `q` sale con **filas = tiempo**: la postura k es `q(k,:)`.
No verificar	CD de la solución → tiene que dar el punto pedido.